Reflection: Why LIFO in the Delimiter Checker and Postfix Evaluator

Eddison, CMPS2131, Lab 2: Stacks and Expression Evaluation

Have you ever wondered why a stack, out of all data structures, is the right tool for checking delimiters and evaluating postfix expressions? Both check_delimiters and eval_postfix in this lab rely on a stack because each one has to remember something for a little while and then act on the most recent thing it remembered, not the oldest. In the delimiter checker, each element on the stack is an opening delimiter that has not been closed yet, a '(', '[', or '{' still waiting for its partner. Pushing an opener means the expression has a piece that is still open, and popping means a closer has just been found, so it needs to be checked against whatever opener is still open. In the postfix evaluator, each element on the stack is an operand value that has been computed but not yet used by an operator, either a number straight from the input or the result of an earlier calculation. Pushing a number means there is a value waiting to be used, and when an operator shows up, popping twice retrieves the two operands it needs, while pushing the result puts a new waiting value back on the stack for whatever comes next.

Both problems have a nesting structure, and nesting resolves from the inside out. The innermost, most recently opened piece is always the first one that gets closed or finished, and that is exactly what LIFO provides. Moreover, stack_push and stack_pop in stack.c both operate on stack->top, so the last value stored is always the first one retrieved. A queue would hand back the oldest item instead, which is the opposite of what either algorithm actually needs. Consider the expression ([{}]). Scanning left to right pushes '(', then '[', then '{', so the stack from bottom to top holds (, [, {. The very next character is '}', a closer. If the check used the oldest opener instead of the most recent one, it would compare '}' against '(', the first thing pushed, and wrongly report a mismatch even though the expression is perfectly balanced. Using the most recently pushed opener is what lets '}' correctly match the '{' that was opened right before it. Only after that inner pair is resolved does the next closer, ']', become responsible for matching '[', and finally ')' matches '('. Popping the top of the stack each time is what walks the closers back through the openers in the reverse order they were opened, which is the only order that actually reflects how the delimiters are nested in the text.

The postfix evaluator needs that same discipline for the same reason. Consider the postfix expression 5 1 2 + 4 * + 3 -, which represents 5 + ((1 + 2) * 4) minus 3. Tokens 5, 1, and 2 are pushed in order, so the stack from bottom to top becomes 5, 1, 2. When '+' arrives, it must combine 1 and 2, the two values computed most recently, not 5, which was pushed earlier and does not belong to that sub-expression yet. Popping twice retrieves 2 then 1, giving 1 + 2 = 3, which gets pushed back on top of 5, so the stack becomes 5, 3. Next, '4' is pushed, giving 5, 3, 4. The '*' operator again needs the two most recent values, 3 and 4, not 5, producing 3 times 4 = 12, so the stack becomes 5, 12. The second '+' then combines the two most recent values, 5 and 12, giving 17. However, if the algorithm pulled from the bottom of the stack instead of the top at any of these steps, it would combine values from unrelated parts of the expression and produce a meaningless result. Using the most recently pushed operand each time is what keeps every operator tied to the sub-expression that was just computed for it, which is exactly how postfix notation is meant to be evaluated.

As mentioned before, both algorithms come down to the same idea: whatever was opened or computed last is also the first thing that needs to be closed or used next. That is not just a coincidence of these two problems, it is the defining property of anything with nested, bracket-like structure, whether the nesting is visible, like parentheses, or hidden inside the sub-expressions of a postfix formula. A stack's LIFO discipline mirrors that nesting exactly, which is why it is the natural, and really the only correct, data structure for either task.